

Claude Permissions Explained

FRONTMATTER

title: Claude Permissions Explained
video no: 11 of 30
series: Building an AI Business From Scratch
voice_reference: ../../Input/4. Resources/2.
scripting_resources/voice_reference/nate_herk_voice_reference.md
video_type: concept
hook_type: Problem-Statement
script_versions: A + B
status: DRAFT

REFERENCE VIDEOS

- Configure permissions -- Claude Code Docs (<https://code.claude.com/docs/en/permissions>) -- Anthropic -- Use as ground truth for what permissions actually control and the five permission modes
- Claude Code auto mode: a safer way to skip permissions (<https://www.anthropic.com/engineering/claude-code-auto-mode>) -- Anthropic Engineering -- Watch/read for the auto-mode vs YOLO mode distinction: this is the nuance most videos get wrong
- CLAUDE CODE FULL COURSE 4 HOURS: Build & Sell (2026) (<https://www.youtube.com/watch?v=QoQBzR1NIqI>) -- Nick Saraev -- Watch the permissions section: how Nick balances speed and safety in a real production workflow

THE ONE THING

Permissions control how much Claude Code can do without asking you first -- configure them right and Claude is fast; ignore them and Claude is either painfully slow or quietly dangerous.

HOOK

Claude Code keeps asking your permission for everything. Or it is doing things you wish it had asked about first. Either way, the fix is the same -- and it takes 5 minutes.

TARGET VIEWER

Someone who finds Claude Code either too slow (constant approval prompts) or too risky (Claude touched something it should not have) -- both problems have the same root cause.

PAYOFF

After watching, the viewer knows how to set Claude permissions to match their comfort level, can grant specific safe commands auto-approval, and knows exactly what to never auto-approve.

BEAT STRUCTURE

Beat -- Too Careful or Not Careful Enough

Point: By default, Claude Code asks permission for almost every action. You can dial this up or down -- but most people never touch the settings and live with the wrong default forever.

Visual: Terminal: Claude asking permission for every bash command (slow, frustrating) vs Claude running autonomously on a deletion task (fast, dangerous) -- show both extremes

Target words: ~110w

Beat -- How Permissions Work

Point: Permissions control which tools Claude can use and which commands it can run without prompting. They live in `.claude/settings.local.json` in your project folder.

Visual: `.claude/settings.local.json` file open in editor -- highlight the allowed tools list and the denied commands list

Target words: ~160w

Beat -- The Permission Modes

Point: Five modes from ask-every-time to fully autonomous. The right mode depends on how sensitive your project is and how much you trust the specific task Claude is running.

Visual: Slider diagram: five modes from most conservative to most autonomous with a use case label for each position

Target words: ~190w

Beat -- What to Auto-Approve and What Not To

Point: Safe to auto-approve: read operations, git status, running tests, listing files. Never auto-approve: delete commands, file overwrites, external API calls, anything touching production.

Visual: Two-column list on screen: green checkmarks for safe auto-approve, red X for never -- then `/permissions` command in terminal opening the interactive UI

Target words: ~190w

Beat -- Auto Mode vs YOLO Mode

Point: Auto mode runs Claude autonomously with safety constraints still active. YOLO mode (`--dangerously-skip-permissions`) removes all checks. Use auto mode for real work. YOLO mode only for sandboxed experiments.

Visual: Terminal demo of auto mode in a real project; hard cut to warning graphic when showing YOLO mode; emphasise the isolation requirement

Target words: ~140w

Beat -- CTA Bridge

Point: Permissions configured. Next: Claude Workflows -- how to stop thinking in single prompts and start building with Claude from end to end.

Visual: Preview clip from Claude Workflows Explained

Target words: ~60w

ANALOGY BANK

- Permissions: Like trust settings for a contractor -- a trusted contractor you have worked with for years gets a key to your office. A new one you supervise closely. Claude's permissions work the same: dial up

trust as confidence in the task grows.

- Auto-approving safe commands: Like setting recurring calendar blocks for meetings you have every week -- you still have the meeting, you just remove the friction of re-scheduling it every time from scratch.

THESIS LINE LOCATION

Beat 4 -- The insight is: the permission system is not about distrust, it is about knowing which actions are recoverable (read, test, git status) vs which are not (delete, overwrite, production write) -- that distinction drives everything.

FRAMEWORK PHRASE

Control, not friction

SPECIFICITY BANK

- Settings file: .claude/settings.local.json in project root
- /permissions command launches interactive permissions UI in terminal
- --dangerously-skip-permissions flag skips all permission checks (YOLO mode)
- Auto-mode: Anthropic-recommended safer alternative to YOLO mode
- Five permission modes available, configurable via settings.local.json
- Shift+Tab cycles between three core permission modes during a session

CONSTRAINTS

- Do not explain CLAUDE.md -- that was the previous video
- Do not go into MCP server permissions -- keep it focused on Claude Code core permissions
- Keep it practical: what to do, not the philosophy of AI safety
- Do not make YOLO mode sound appealing -- be explicit about when it is and is not appropriate

CTA DIRECTION

Permissions are sorted. Next: Claude Workflows -- how to stop thinking in single prompts and start using Claude to build things from end to end.

PERSONAL ANGLE [DANIEL TO FILL]

Suggested: A specific moment when Claude modified or deleted something it should not have -- and the exact change you made to permissions that prevented it from ever happening again.

VULNERABILITY MOMENT [DANIEL TO FILL]

Suggested: Running in YOLO mode on a real project -- and what happened. The specific cost of ignoring permission boundaries.

Note: belongs in Beat 5